Training Different Classifiers on the YesNo Dataset (torchaudio)

Student name: Haneen alkadhimi **ID: 404131081**

1. Introduction

The goal of this assignment is to build a simple speech classification system that can tell the difference between two spoken words, yes and no. I used the YesNo dataset that comes with torchaudio and compared several classical machine learning classifiers with a small convolutional neural network. All the work was done in Google Colab using PyTorch, torchaudio and scikit-learn.

2. Dataset

The YesNo dataset contains 60 recordings of one male speaker. Each recording is sampled at 8000 Hz and lasts about six seconds. In every recording the speaker says eight words in Hebrew, where "ken" means yes and "lo" means no. The labels are given as a list of eight values, for example the first file has the labels [0, 0, 0, 0, 1, 1, 1, 1], which means four no words followed by four yes words. The first file has 50,800 samples, which is 6.35 seconds of audio.

Since the labels describe individual words and not whole files, the classification has to be done at the word level. This means each recording must first be split into its eight words.

3. Method

3.1 Word segmentation

When I plotted the waveform of the first recording, the eight words appeared as eight clear bursts separated by silence, with one small noise spike near the end. To find the words automatically I used a simple energy based voice activity detection. The signal was divided into frames of 200 samples (25 ms) with a hop of 80 samples (10 ms), and the energy of each frame was computed as the sum of squared samples. Squaring made the speech much stronger than the background, and the small noise spike became almost invisible in the energy curve.

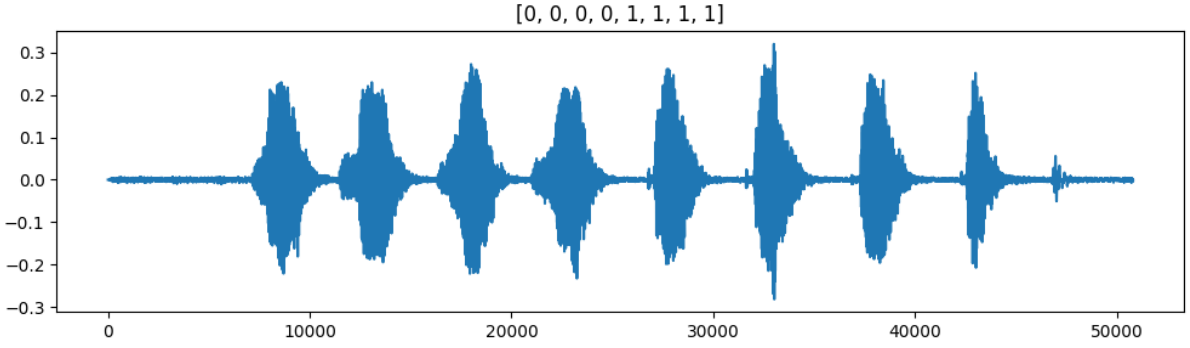


Figure 1. Waveform of the first recording with labels [0, 0, 0, 0, 1, 1, 1, 1]

My first version used a fixed threshold equal to 10% of the maximum energy. It worked well on the first file and found exactly eight words, but when I applied it to all 60 files only 37 of them gave eight segments. Most of the failed files gave nine to eleven segments. Looking at the energy curves, the reason was that some words have two peaks with a dip between them, so a single word was sometimes cut in two (the double peak of the fourth word can be seen in Figure 2), and a few short noises were also counted as words.

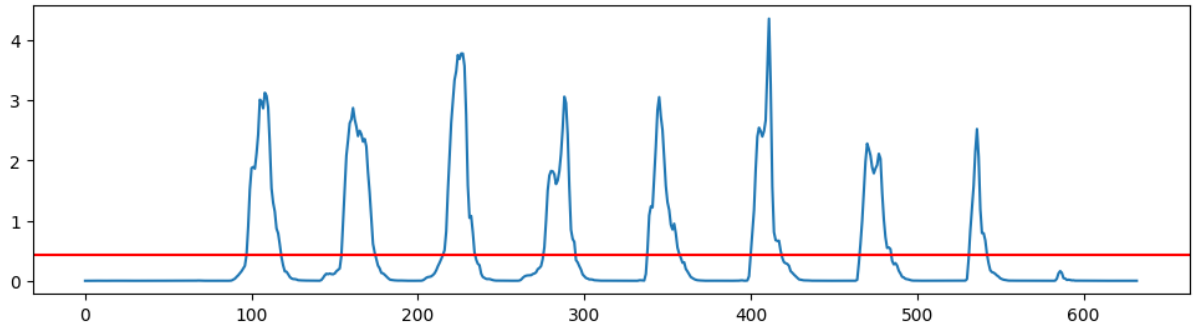


Figure 2. Frame energy of the first recording and the 10% threshold (red line)

To fix this I made three changes. First, segments that are closer than 100 ms to each other are merged into one word. Second, segments shorter than 100 ms are removed as noise. Third, instead of a fixed threshold, the function tries 60 thresholds from 30% down to 1% of the maximum energy and stops at the first one that gives exactly eight words. With these changes 59 out of 60 files were segmented correctly. File number 21 still gave seven segments, probably because two words were spoken too close together, so I excluded it. This left 472 words in total, 249 yes and 223 no, so the classes are fairly balanced. The word lengths ranged from 0.10 to 0.34 seconds with a mean of 0.17 seconds.

3.2 Feature extraction

For the classical classifiers each word was described by MFCC features. I computed 13 MFCC coefficients per frame (n_fft = 256, hop length = 80, 40 mel bands) together with their delta coefficients, which describe how the MFCCs change over time. Because the words have different lengths, I took the mean and the standard deviation of each coefficient over time. This gives a fixed vector of 52 values for every word (13 MFCC + 13 delta, times mean and standard deviation). The final feature matrix had the shape 472 x 52.

3.3 Classifiers

Seven classifiers from scikit-learn were trained on the MFCC features: Logistic Regression, SVM with an RBF kernel, K-Nearest Neighbors with k = 5, Decision Tree, Random Forest with 200 trees, Gaussian Naive Bayes and a Multi-Layer Perceptron with two hidden layers (64 and 32 units). Each classifier was placed in a pipeline with a StandardScaler, so the scaling is always learned from the training data only.

In addition, I trained a small CNN on log-mel spectrograms. Every word was padded with zeros or cut to 3200 samples (0.4 s), which is longer than the longest word, and converted to a 40 x 41 log-mel spectrogram. When comparing the spectrograms of a no word and a yes word, the no word had most of its energy in the low mel bands, while the yes word also showed a clear band in the higher frequencies. This matches the vowels of the two words, since the "e" in ken has higher resonances than the "o" in lo. The network has three convolution blocks (16, 32 and 64 filters, each with batch normalization, ReLU and pooling), followed by global average pooling, dropout of 0.3 and a linear layer with two outputs. It was trained for 30 epochs with the Adam optimizer, a learning rate of 0.001 and a batch size of 32.

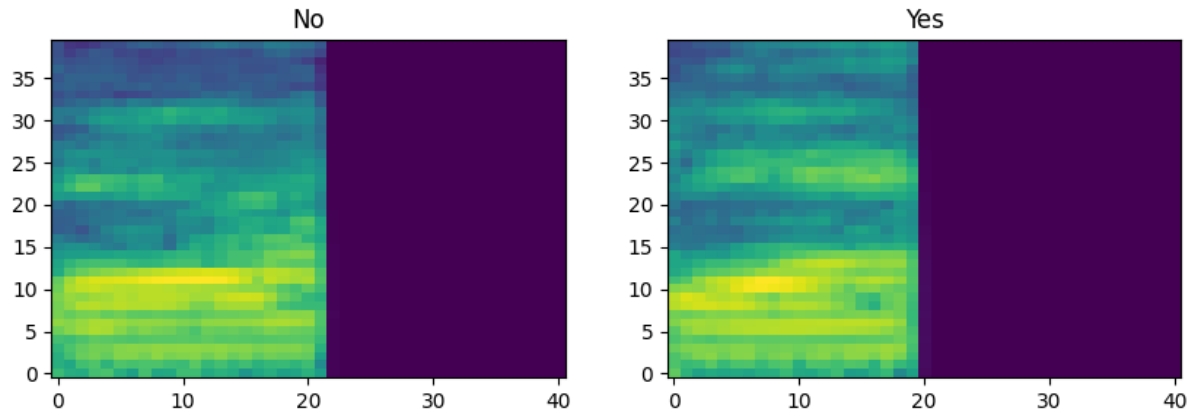


Figure 3. Log-mel spectrograms of a no word (left) and a yes word (right)

3.4 Evaluation

An important point is that words from the same recording are very similar to each other. If the data is split at the word level, words from one file can appear in both the training and the test sets, which would make the results look better than they really are. For this reason all splits were done by recording. For the main test I used GroupShuffleSplit with 80% of the files for training and 20% for testing, which gave 47 training files (376 words) and 12 test files (96 words). I also ran 5-fold GroupKFold cross validation on all the data, because a single split can be lucky or unlucky. Accuracy and F1 score were used as metrics.

4. Results

Table 1 shows the results of all eight models, and Figure 4 compares the classical classifiers.

Model	Test Accuracy	Test F1	CV Accuracy
Logistic Regression	1.000	1.000	1.000
SVM (RBF)	1.000	1.000	1.000
KNN (k=5)	1.000	1.000	1.000
Random Forest	1.000	1.000	1.000
Naive Bayes	1.000	1.000	1.000

MLP	**1.000**	**1.000**	**1.000**
CNN (log-mel)	**1.000**	**1.000**	**0.996**
Decision Tree	**1.000**	**1.000**	**0.994**

Table 1. Test and 5-fold cross validation results

All models classified the 96 test words without any error. In cross validation, six of the classical models also reached 100%. The Decision Tree dropped slightly to 99.4% and the CNN to 99.6%, which corresponds to only one or two misclassified words out of 472.

Figure 4. Test and cross validation accuracy of the classical classifiers

The CNN training loss started at about 0.69, which is the value expected when a binary classifier is guessing randomly (ln 2). It fell quickly to 0.247 after 5 epochs and 0.069 after 10 epochs, and then levelled off, reaching 0.014 at epoch 30. The curve was smooth without jumps, which shows the training was stable and that 30 epochs were enough.

Epoch	Training Loss
5	**0.2468**
10	**0.0687**
15	**0.0411**
20	**0.0254**
25	**0.0159**
30	**0.0139**

Table 2. CNN training loss

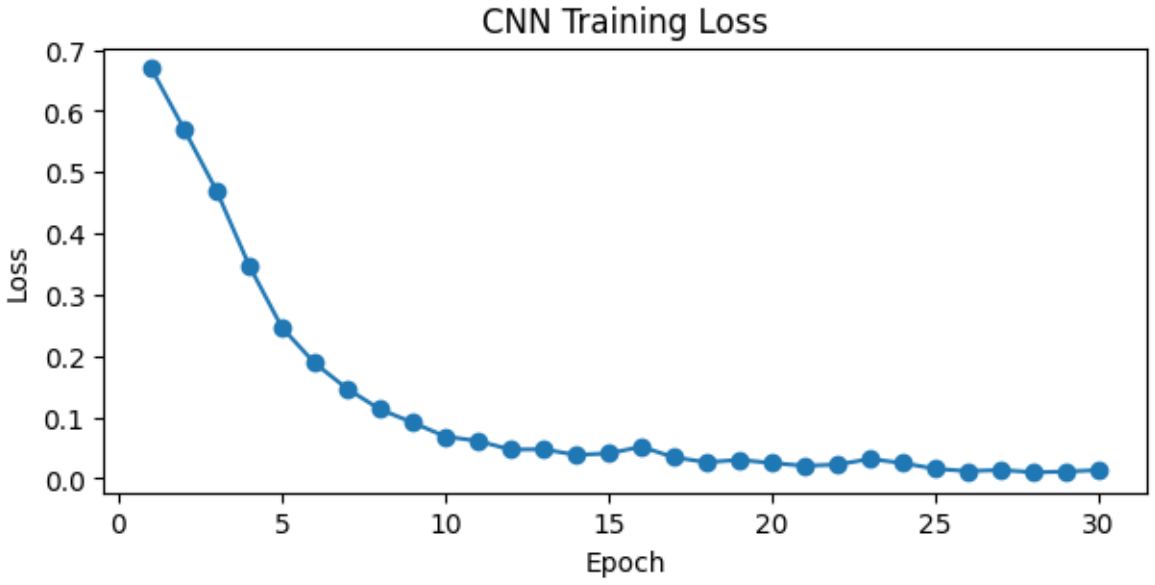


Figure 5. CNN training loss over 30 epochs

5. Discussion

The very high accuracy is not a sign of a mistake, because the split was done by recording and the test files were never seen during training. The task is simply easy: there is only one speaker, the recordings are clean, and the two words sound very different. The MFCC features already capture this difference well, so even simple linear models like Logistic Regression can separate the two classes perfectly.

Because the test accuracy was 100% for every model, it could not show any difference between them. Cross validation was more useful here. For example, the Decision Tree looked perfect on the single test split, but cross validation showed that it is a little weaker than the others. A single tree makes hard decisions on one feature at a time, so small changes in the data can affect it. Random Forest avoids this by averaging 200 trees, and it reached 100%.

The CNN did not do better than the classical models, and was even slightly behind them. I think there are two reasons for this. The dataset is very small (472 words), while deep networks usually need much more data to show their advantage. Also, the MFCC features are designed by hand based on knowledge of human hearing, so they give the classical models useful information directly, while the CNN has to learn everything from the spectrogram. For a simple task with little data, a classical model with good features is enough, and it is also faster to train and easier to explain.

The most difficult part of the work was the segmentation, not the classification. The fixed threshold failed on more than a third of the files, and the final method with merging, minimum length and an adaptive threshold was needed to reach 59 out of 60.

6. Conclusion

In this assignment I built a full pipeline for yes and no word classification on the YesNo dataset: energy based segmentation, MFCC feature extraction, seven classical classifiers and a CNN. Almost all models reached 100% accuracy when evaluated with a split by recording, with the Decision Tree (99.4%) and the CNN (99.6%) slightly lower in cross validation. The results show that for this dataset the quality of the segmentation and features matters more than the choice of classifier. A natural next step would be to test the models on noisy audio or on more speakers, where the differences between the classifiers should become clearer.